CAPSTONE PROJECT · PYTHON · FH DORTMUND

AI-Powered Smart Energy Management System

Subsystem: Real-Time Energy Monitoring & Sustainability Analytics

Nasim Abhari · Matr. no. 7226039

Prof. Galyna Tabunshchyk · November 2026



ROLES DISTRIBUTION

One student, every role, one complete subsystem

The group started with four members and four subsystems. The other three left the course, and the professor approved that I finish **one subsystem alone**.

I chose the subsystem that covers the topic's key features: real-time monitoring, the dashboard and the sustainability score.

Role	Responsible
Requirements & domain analysis	Nasim Abhari
Architecture & UML	Nasim Abhari
Python development	Nasim Abhari
Database & Docker	Nasim Abhari
Testing	Nasim Abhari
Documentation & video	Nasim Abhari

THE DOMAIN

Buildings waste energy because nobody sees it in time

≈ 40 %

of the EU's energy is used by buildings

Source: European Commission, Energy Performance of Buildings

Problem

Monthly bills arrive too late. HVAC runs in empty rooms, and loads pile up in the expensive evening peak.

Users

Facility managers of offices and universities, and home owners with solar and battery storage.

Goal

Lower energy use, cost and CO_2 while keeping comfort, using live data and AI advice.

REQUIREMENTS

10 functional requirements

ID	The system shall …
FR1	register and delete buildings (office, university, home) with area and city
FR2	install default smart meters per building: HVAC, lighting, plug loads, solar, battery
FR3	accept meter readings in real time through a JSON REST API (single or batch)
FR4	filter out readings of offline meters and sub-threshold readings
FR5	convert every reading into kWh, kg CO_2, cost in € and a peak-tariff flag
FR6	store buildings, meters, readings and AI advice in a relational database
FR7	show consumption, solar, cost, savings and CO_2 reduction for 1 / 7 / 30 days
FR8	show live meter power and the net grid load, updated every 3 seconds
FR9	calculate a sustainability score 0–100 and a grade A–E with breakdown
FR10	give AI recommendations (Gemini or OpenAI), answer questions and keep the history

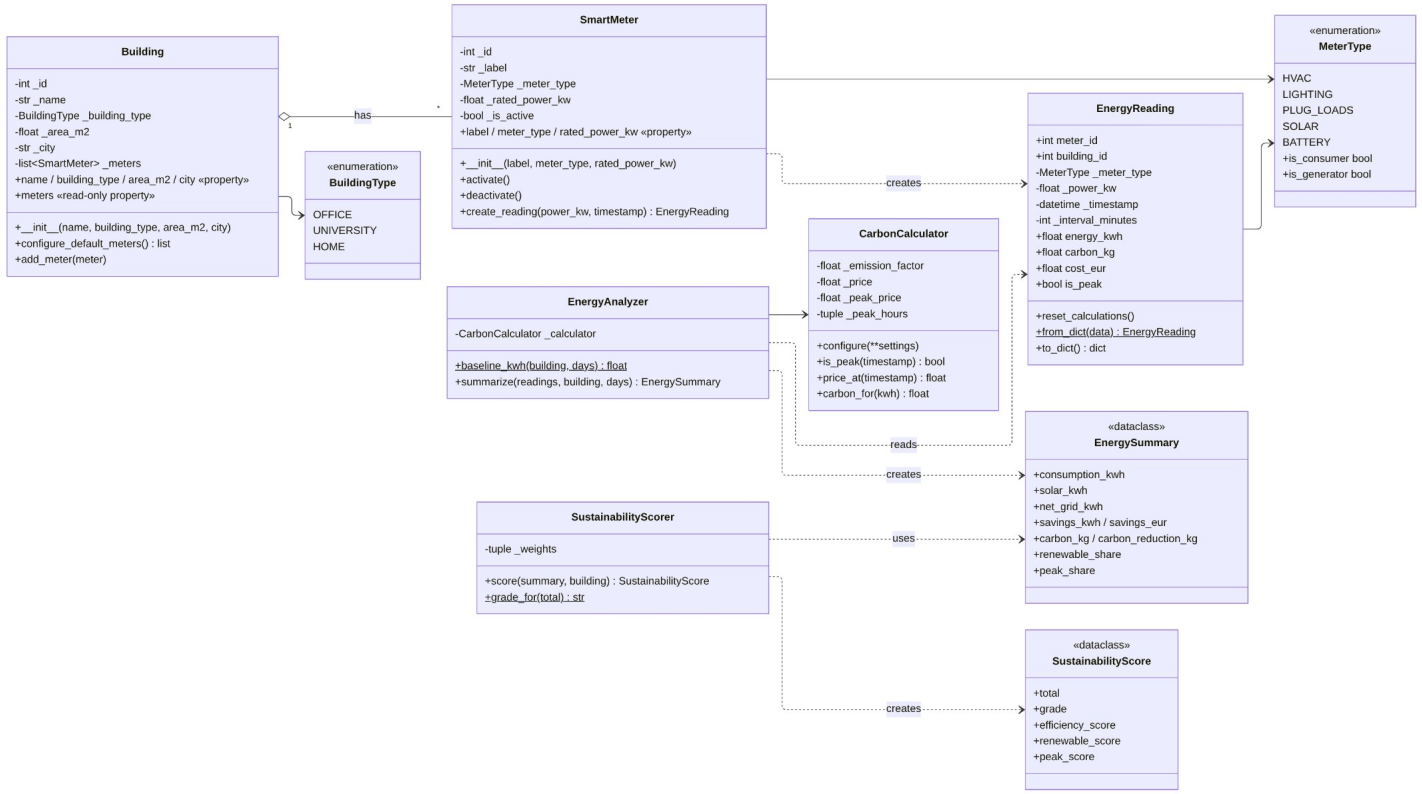
REQUIREMENTS

10 non-functional requirements

ID	Quality	Requirement → how it is met
NFR1	Usability	all key figures on one screen, units always shown
NFR2	Performance	dashboard < 1 s, live tick < 300 ms → measured ≈ 12 ms / 7 ms
NFR3	Reliability	works without the AI service → rule-based fallback coach
NFR4	Robustness	bad input never crashes → correct HTTP code + JSON error
NFR5	Maintainability	layered architecture, PEP 8, docstrings in every module
NFR6	Testability	≥ 15 automated tests → 40 tests, 93 % coverage
NFR7	Portability	Docker + MySQL or local SQLite, only DATABASE_URL changes
NFR8	Security	secrets in .env, ORM only (no SQL injection), no leaked errors
NFR9	Extensibility	a new pipeline step = one subclass, plugged in with |
NFR10	Availability	Bootstrap and Chart.js bundled → dashboard works offline

UML CLASS DIAGRAM · 1 OF 3 · DOMAIN & ANALYTICS

A building owns meters; meters create readings



10 classes

3 domain, 3 analytics, 2 enums, 2 result dataclasses

Encapsulation

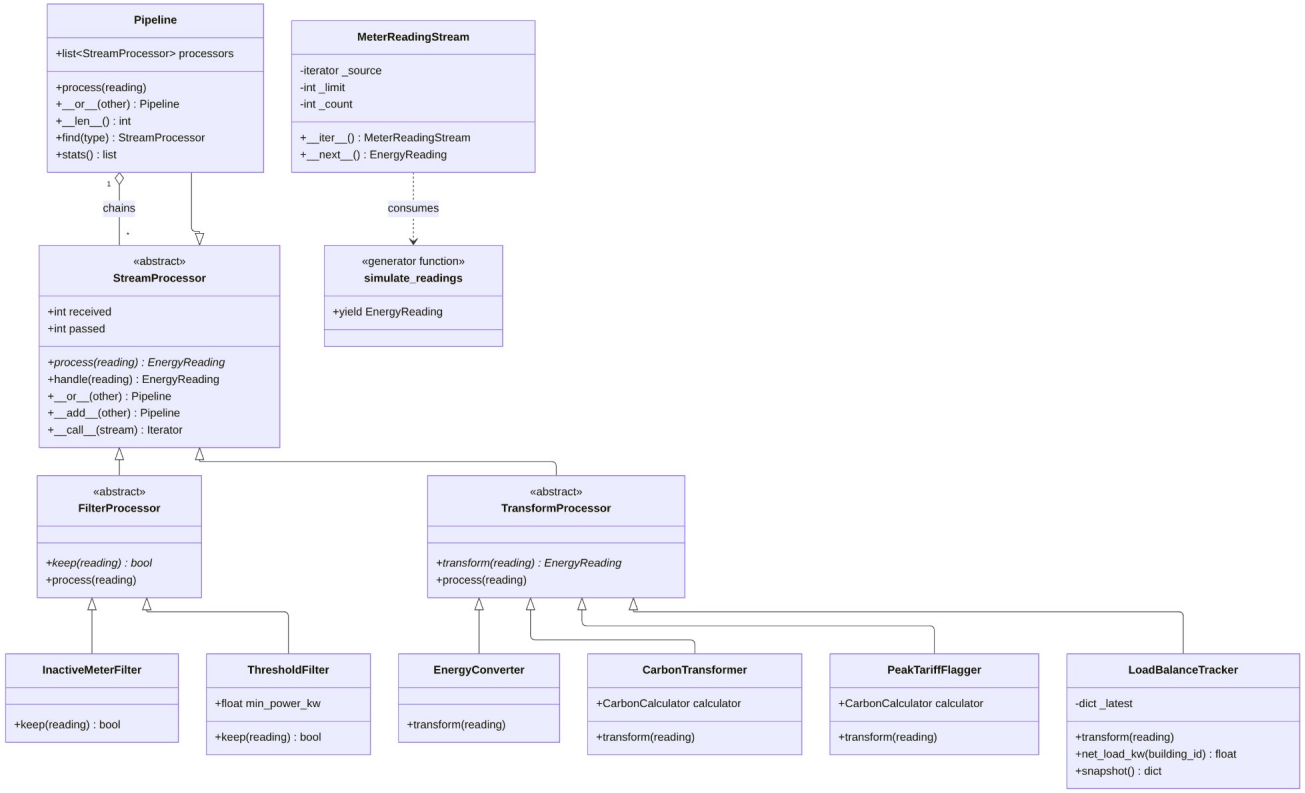
Private attributes, @property getters, setters that reject invalid values

Setup methods

configure_default_meters(), activate(), from_dict(), configure()

UML CLASS DIAGRAM · 2 OF 3 · STREAM PIPELINE

Abstract processors, chained with the | operator



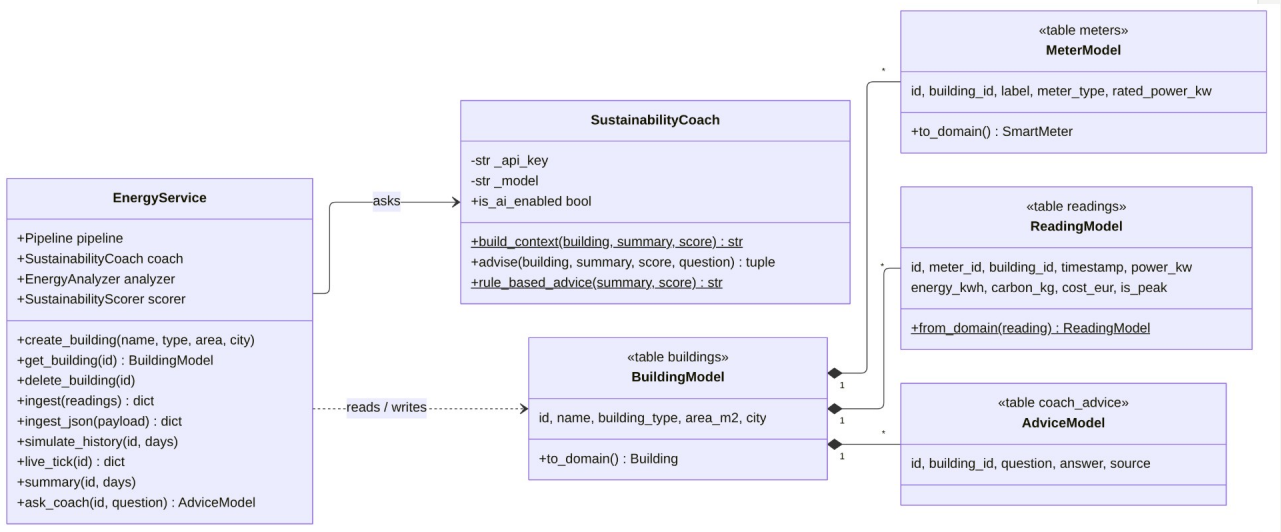

```
pipeline = (
InactiveMeterFilter()
| ThresholdFilter(0.05)
| EnergyConverter()
| CarbonTransformer(calc)
| PeakTariffFlagger(calc)
| LoadBalanceTracker())
```

ABCs with @abstractmethod · **polymorphism** via handle() · **__or__ / __add__** · **iterator** MeterReadingStream · **generator** simulate_readings()

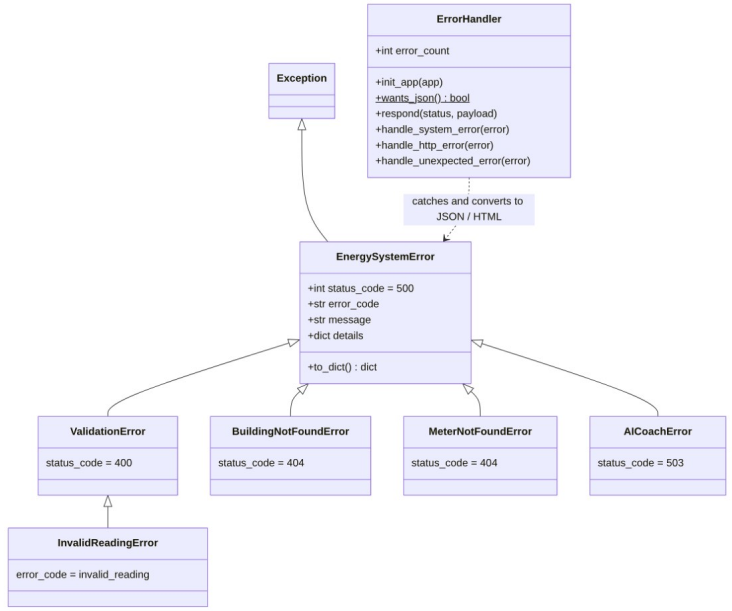
UML CLASS DIAGRAM · 3 OF 3 · WEB, DATABASE, AI & ERRORS

One service connects the web, the database and the AI

EnergyService · SustainabilityCoach · ORM models

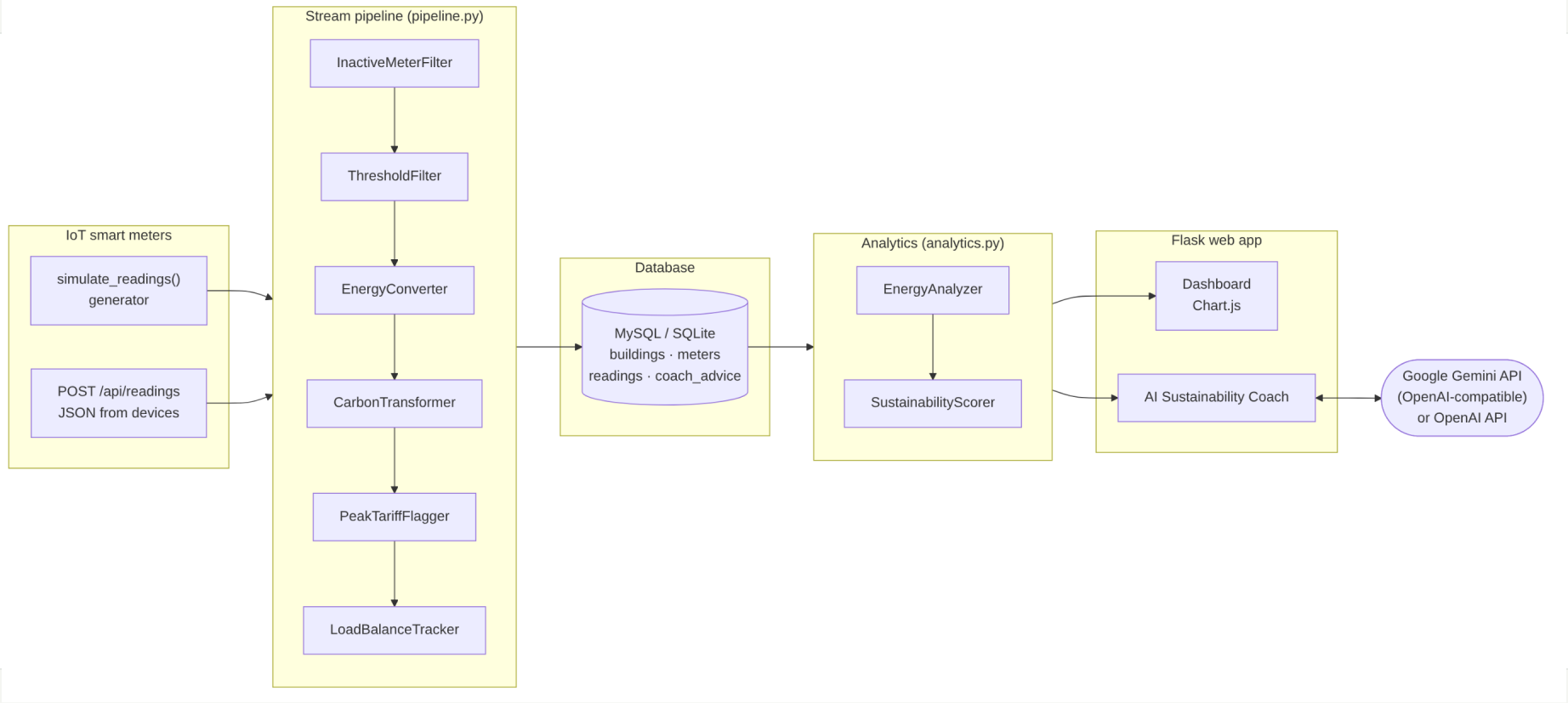


Error handling (HA4)



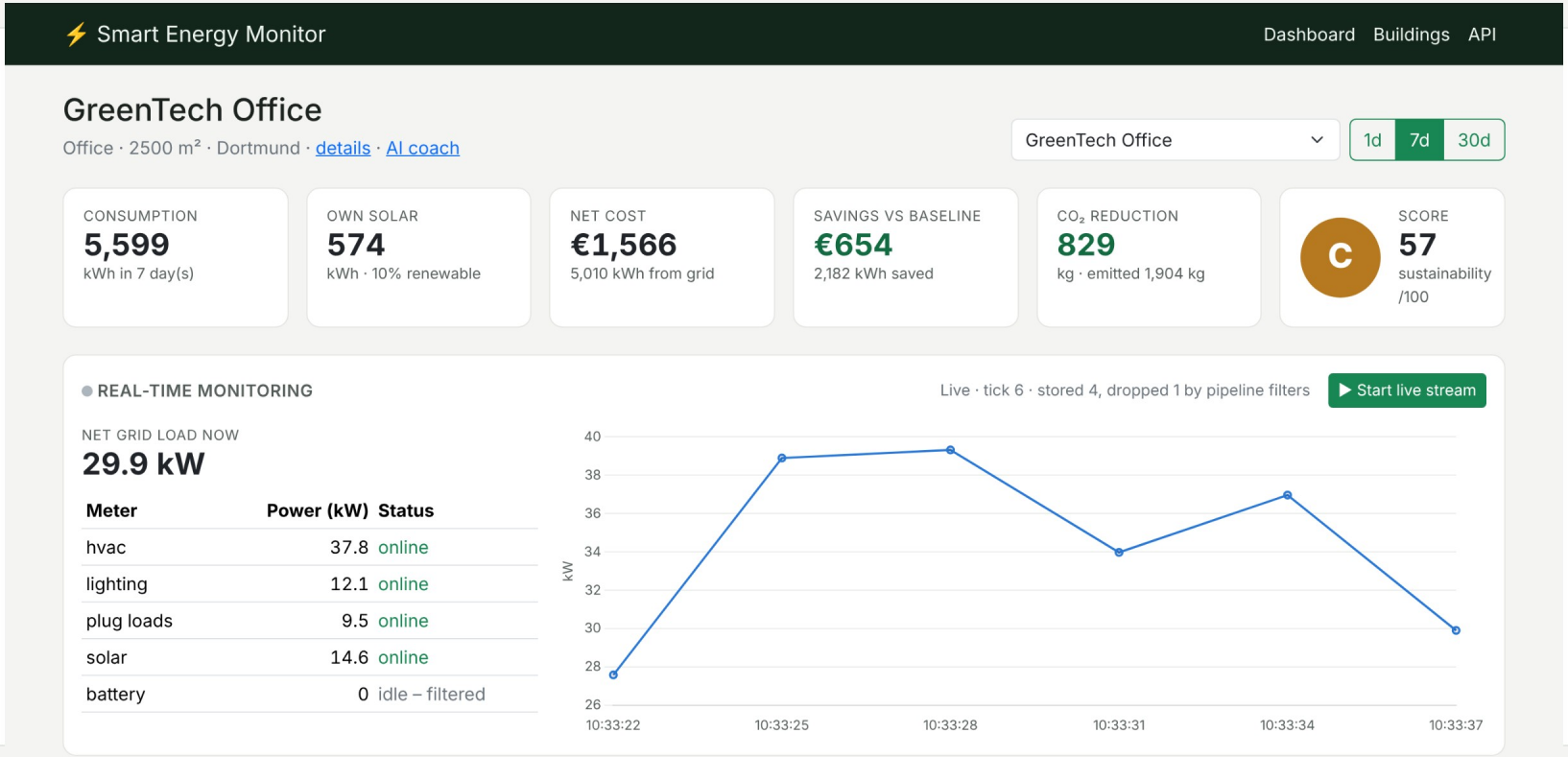
HOW IT WORKS · ARCHITECTURE

Meter → pipeline → MySQL → analytics → dashboard & AI



HOW IT WORKS · DASHBOARD

Usage, savings, CO_2 and score at a glance, updated live



HOW IT WORKS · SUSTAINABILITY SCORE

Three measurable parts, weighted into one grade

50 %

Efficiency

kWh per m² per year vs. benchmark (office 150, university 180, home 120)

30 %

Renewables

Own solar ÷ consumption; 50 % or more gives full points

20 %

Peak load

Share used 17–21 h; 10 % or less gives full points, 40 % gives zero

GreenTech Office: 0.5 × 72 + 0.3 × 20 + 0.2 × 76 ≈ **58** → **grade C**

A ≥ 85 · B ≥ 70 · C ≥ 55 · D ≥ 40 · E < 40

HOW IT WORKS · AI SUSTAINABILITY COACH

Google Gemini turns the building's own numbers into advice

how can we lower our overall costs?

Gemini · 12.11.2026 17:10

- Optimize HVAC operation, as it accounts for 68.7% (13,628 kWh) of total energy use; lowering setpoints by 1–2°C and reducing ventilation during non-lecture hours will significantly trim the €5,942 weekly cost.
- Leverage battery storage to shift peak-tariff consumption (currently 22%); the battery used only 1.7 kWh this period and should be charged off-peak to lower electricity rates.
- Reduce lighting (3,280 kWh) and plug load (2,935 kWh) baselines across the 8,000 m² area by installing occupancy sensors and automated off-hour shut-off switches.
- Align heavy campus operational loads with solar production windows to increase your renewable self-consumption ratio above the current 7% (1,301 kWh) and directly decrease grid purchases.

1 · Context

The prompt carries the building's kWh, solar share, cost, CO_2, peak share and score

2 · Advice

3–5 concrete steps from Gemini, or an answer to the manager's own question

3 · Always works

Same OpenAI client for Gemini or OpenAI; API down → rule-based coach

UNIT TEST RESULTS

All 40 test cases pass, with 93 % code coverage

40 / 40
passed in ≈ 0.5 s

93 %
line coverage (pytest-cov)

File	Cases	What it checks
test_domain.py	16	setters reject bad values, meters scale with area, peak tariff, grades, savings
test_pipeline.py	6	ABCs, | and +, filters, transforms, iterator, generator
test_web.py	18	pages, API, database, 404/400/500 errors, AI coach

Each test gets a fresh **in-memory database**. The AI client is replaced by a **FakeOpenAI** client, so tests run offline, fast and free, including the “API is down” case.

CONCLUSION

Every checklist item is delivered in one Python MVP

Done

- ✓ Web interface (Flask)
- ✓ Data analysis & visualisation
- ✓ Database storage (MySQL)
- ✓ AI Sustainability Coach (Gemini)
- ✓ Testing: 40 passed, 93 %
- ✓ Docker

Next steps

Real meters over MQTT instead of the simulator

Machine-learning load forecast to pre-heat before the peak

User accounts and alerts for unusual consumption

Thank you – questions?